

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 1
Revisit of Programming Design

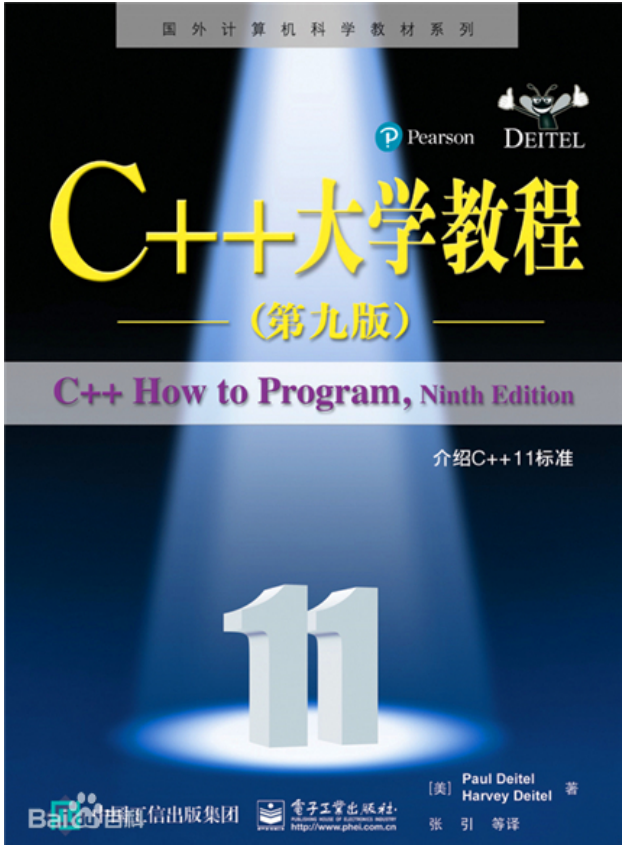
Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Outline：

- Class and Object
- Operator Overloading
- Inheritance
- Polymorphism
- Stream
- Templates
- Exceptions
- Standard Template Library



Class and Object

```
class Name {
public:
    data fields
    functions
    Name()
private:
    data fields
    functions
};
```

1. Capitalize the initials

```
class Circle
{
public:
  // The radius of this circle
  double radius;

  // Construct a circle object
  Circle()
  {
    radius = 1;
  }

  // Construct a circle object
  Circle(double newRadius)
  {
    radius = newRadius;
  }

  // Return the area of this circle
  double getArea()
  {
```

Data field

Constructors

Function

of class name
2. No initial value when declaring data fields

```
    return radius * radius * 3.14159;
  }
};
```


Class and Object

- Access Authority

  Public - accessible wherever the program has access to an object of class Time

  Private - accessible only to member functions of the class

  Protected – accessible to member functions of the class and the derived classes
- "public" and "private" can appear any times in any order in a class
- Data fields and functions can be declared in any order in a class

Class and Object

- Constructor
  - A special type of function to construct objects from the class
  - The constructor is invoked when an object is created
  - Can be overloaded
  - No-arg constructor
  - Default construct
  - Copy constructor (for performing deep copy, note “=“ operator)



- Destructor
  - The destructors are automatically called when objects are destroyed

Class and Object

- Setter and Getter functions (*principle of least privilege*)
- Utility functions
- The “this” pointer
  - A special built-in pointer that points to the current object
  - Implicitly reference member data and functions
  - Static member functions have no “this” pointers.
  - Cascaded member function calls

- Use new/delete to create/destroy objects

Class and Object

- Keywords: “static”, “const”
  - Static/const functions and static/const variables
  - How to initialize static/const variables?
  - Use member initializer
- Keyword “friend”
  - Can access private and protected members of a class

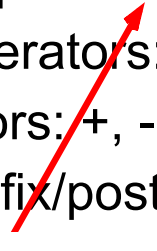
Class and Object

- Separating Interface from Implementation
  - Split into header files and source files
  - Using the *preprocessor directives* (预处理指令) “#ifndef” and “define” to prevent multiple declaration
  - Note template classes

- Information hiding
- Encapsulation
- Abstract Data Type (ADT)

Operator Overloading

- Overloading Assignment Operator: =
- Overloading Arithmetical Operators: +, -, *, /
- Overloading Shorthand Operators: +-, -=, *=, /=
- Overloading Unary Operators: +, -, !
- Overloading ++ and --, (prefix/postfix)



- Overloading Subscript Operator: [] (lvalue/rvalue)
- Overloading Comparison Operators: ==, <, >, <=, >=, !=

Operator Overloading

- Overloading Stream-Insertion and Stream-Extraction Operators: <<, >>
- Cast Operator (type conversion)



- *Classname::operator type();*

- Operator functions as class members or as friend functions

	Friend function	Member Function
a + b	operator+(a,b)	a.operator+(b)
a++	operator++(a,0)	a.operator++(0)
–a	operator–(a)	a.operator–( )

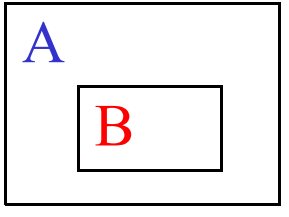
Inheritance

- The concepts of “is-a”, “has-a”, “use-a” relations.
- Inheritance

- New classes created from existing classes
- Absorb attributes and behaviors
- Base and derived classes

```
class DerivedClass: accessControl BaseClass{…};
```

```
class A: public B{
public:
…
private:
…
};
```



Inheritance

- Access control:

基类（父类）的访问特性	类的继承特性	派生类（子类）的访问特性
Public Protected Private	Public	Public Protected No access
Public Protected Private	Protected	Protected Protected No access
Public Protected Private	Private	Private Private No access

- Single inheritance and multiple inheritance

Inheritance

- Method hiding (Redefine): To shield/hide the original function
- Constructor and destructor in inheritance
  - Use member initializer syntax
  - Note the order of constructor calls
  - Destructors are called in the reverse order of constructor calls
- Type conversion
  - baseClassObject = derivedClassObject; (OK)
  - derivedClassObject = baseClassObject; (need to overload the assignment operator)
  - *baseClassPointer = &derivedClassObject; (OK)
  - *derivedClassPointer = &baseClassObject; (Compile error)

Polymorphism

- Concept of Polymorphism:
  - Ability for objects of different classes to respond differently to the same function call
  - Base-class pointer (or reference) calls a virtual function
  - C++ chooses the correct overridden function in object
- Overriding (“virtual” function)
  - To redefine a virtual function in the derived class

Polymorphism

- Pure virtual function
  - Abstract class
  - Define one or more virtual functions as “pure” by initializing the function to zero

  e.g., virtual int func() = 0;
- Virtual destructors
  - If base-class pointer to a derived object is deleted, the base-class destructor will call if base-class destructor is not virtual.

Polymorphism



- Early/static binding
- Late/dynamic binding (vtable)
- Virtual base class (virtual inheritance): to avoid ambiguity in the multiple inheritance.

Stream

- iostream, ffstream, sstream
- Condition state of a stream

s.eof()	如果设置了流 s 的 eofbit 值，则该函数返回 true
s.fail()	如果设置了流 s 的 failbit 值，则该函数返回 true
s.bad()	如果设置了流 s 的 badbit 值，则该函数返回 true
s.good()	如果流 s 处于有效状态，则该函数返回 true
s.clear()	将流 s 中的所有状态值都重设为有效状态

- Strea[illegible]

Stream

- fstream.h: ifstream, ofstream, fstream

ifstream in(“in.txt”);

ofstream out(“file1”, ofstream::out | ofstream::trunc)

in	打开文件做读操作
out	打开文件做写操作
app	在每次写之前找到文件尾
ate	打开文件后立即将文件定位在文件尾
trunc	打开文件时清空已存在的文件流
binary	以二进制模式进行 IO 操作

Stream

- sstream.h: istringstream, ostringstream, stringstream
- It is convenient to use stringstream to parse paragraphs

stringstream strm;	创建自由的stringstream对象
stringstream strm(s);	创建存储s的副本的stringstream对象，其中s是string类型的对象
strm.str();	返回strm中存储的string类型对象
strm.str(s);	将string类型的s复制给strm，返回void

Stream

- Formatting Output

iostream中定义的操纵符(2)		
	internal	在符号和值之间增加填充字符
	fixed	用小数形式显示浮点数
	scientific	用科学记数法显示浮点数
	flush	刷新 ostream 缓冲区
	ends	插入空字符，然后刷新 ostream 缓冲区
	endl	插入换行符，然后刷新 ostream 缓冲区
	unitbuf	在每个输出操作之后刷新缓冲区

iomanip中定义的操纵符	
setfill(ch)	用ch填充空白
setprecision (n)	将浮点精度置为0
setw(w)	读写w个字符的值
setbase(b)	按基数b输出整数

x	nounitbuf	恢复常规缓冲区刷新
x	skipws	为输入操作符跳过空白
	noskipws	不为输入操作符跳过空白
	ws	“吃掉”空白
注：带x的是默认流状态		

Templates

- Generic Programming
- Function templates and class templates

```
template <class or typename T>
returnType functionName(parameterList) {
        //definition
}

template <class T>
class ClassName{
  T var;
```

```
    // other definitions ...
};
```

- Explicitly and Implicitly template function calls.

Templates

- Template and inheritance
- Class template specialization

```
// class template:
template <class T>
class specTemplate  {
  T m_var;
public:
```

```
// class template specialization:
template <>
class specTemplate <char>  {
  char m_var;
public:
```

```
  specTemplate (T inData)
  { m_var = inData; }
  T increase () { return ++m_var; }
};
```

```
specTemplate (char arg) { m_var = arg; }
char upperCase ()  {
  if ((m_var >= 'a') && (m_var <= 'z'))
    m_var += 'A'-'a';
  return m_var;  } } ;
```


Exceptions

- Concept
  - **Exceptions** indicate problems that occur during a program's execution, and the problems are special that they are unexpected and occur infrequently.
  - **Exception handling** means resolve exceptions and make the

programs robust and fault-tolerant.

- Exception Handling Mechanism
  - throwing an exception
  - handling the exception

Exceptions

- Using try-throw-catch
- Multiple catches (note the matching principle for “catch” handler)

```
int main(){
 cout <<"Enter two integers: ";
 int num1, num2;
```

Exception Handler

```
cin >> num1 >> num2;
try{
   if (num2 == 0) throw num1;
   cout << num1 << " / " << num2 << " is "
          << (num1 / num2) << endl;
}catch (int e){
   cout << "Exception: an integer " << e <<
        " cannot be divided by zero" << endl;
}
cout << "Execution continues ..." << endl;
}
```


Exceptions

- Custom Exception classes
  - Use standard classes whenever possible
  - Create yours if necessary

- An exception class is just like any C++ class
- It is often desirable to derive it from exception
  - Directly or indirectly
  - To utilize the common features (e.g., the what() function) in the exception class.

```
// Class DividByZeroException to be used in exception handling
for throwing an exception on a division by zero
class DivideByZeroException {
public:
  DivideByZeroException():
  message("attempted to divide by zero") {}
const char* what() const { return message ; }
private:
  const char* message ;
} ;
```

```
// 使用示例
try {
  throw DivideByZeroException() ;
  //调用类DivideByZeroException的构造函数，
  //实例化类DividByZeroException的一个对象，并将其抛出
} catch ( DivideByZeroException ex) //捕获类DividByZeroException的对象
{
  cout << "Exception occurrs: " << ex.what() << endl ; //调用成员函数
}
```


Standard Template Library

- STL containers

顺序容器（Sequential Containers）	
vector	支持快速随机访问

关联容器（Associative Containers	
map	关联数组：元素通过键来存

list	支持快速插入/删除
deque	双端队列
顺序容器适配器(Sequential Container Adaptors)	
stack	后进先出（LIFO）堆栈
queue	先进先出（FIFO）队列
priority_queue	有优先级管理的队列

	储和读取
set	大小可变的集合，支持通过键实现的快速读取
multimap	支持同一个键多次出现的 map 类型
multiset	支持同一个键多次出现的 set 类型

Standard Template Library

- STL iterators
  - Iterators provide access to objects in the containers yet hide the internal structure of the

container.

- An iterator is like a built-in pointer that can manipulate the elements in a container.
- Different containers may have different types of iterators
- Constant, Mutable, Reverse Iterators

Standard Template Library

- Iterator Types Supported by Containers

STL Container	Type of Iterators Supported
vector	random access iterators
deque	random access iterators
list	bidirectional iterators
set	bidirectional iterators
multiset	bidirectional iterators
map	bidirectional iterators
multimap	bidirectional iterators
stack	no iterator support
queue	no iterator support
priority_queue	no iterator support

Standard Template Library

- STL Algorithms
  - About 80 algorithms in STL
  - Four groups: Nonmodifying Algorithms, Modifying Algorithms, Numeric Algorith

```
template <typename RandomAccessIterator>
void sort(RandomAccessIterator beg, RandomAccessIterator end)

template <typename RandomAccessIterator, typename relationalOperator>
void sort(RandomAccessIterator beg, RandomAccessIterator end, relationalOperator op)
```

```
template <typename ForwardIterator, typename T>
bool binary_search(ForwardIterator beg, ForwardIterator end, const T &value)

template <typename ForwardIterator, typename T, typename
        strickWeakOrdering>
bool binary_search(ForwardIterator beg, ForwardIterator end, const T &value,
        strickWeakOrdering op)
```

Thank you!

